

A Comparative Analysis of Ensemble and Individual Machine Learning Architectures for Mitigating Phishing Attack Risks

1st Arham Siddiqui
Santa Clara High School
Santa Clara, California
arham.siddiqui61@gmail.com

Abstract—Phishing attacks are a pervasive threat to cybersecurity, leaving those with minimal digital literacy susceptible to hacking or identity theft. Such attacks generally come in the form of emails, disguised as potential opportunities for the unsuspecting user, asking for the user to follow a link. These emails mimic or pose as authentic companies/organizations by the use of character substitution, special characters, or extensive characters in their domain name. This issue prompts the development of effective detection mechanisms. In this research paper, we present a comparative analysis of the viability of three machine learning models, namely K-Nearest Neighbors (KNN), Random Forest Classifier (RFC), and an Ensemble model comprised of a Decision Tree, SVC, and RFC for detecting phishing attacks. We utilized an extensive dataset of 30 features with 11,055 data points each. Accuracy, precision, recall, and f1 scores were the metrics used to evaluate performance for the models used. The primary objective of this study is to determine which model offers the most effective performance in detecting such attacks, and analyze ways to optimize the efficiency of said method.

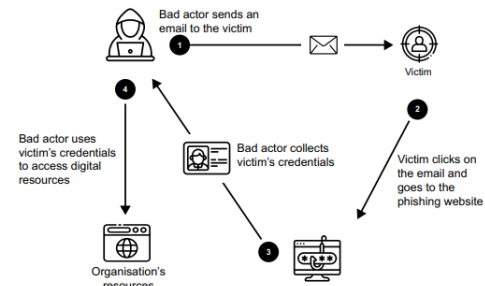
I. INTRODUCTION

Phishing attacks are a cybersecurity threat that come in the form of emails posing as an authentic and trustable entity/organization. Fundamentally deceptive, these attacks are created to trick victims into thinking they have received a potential opportunity, which ultimately leads them to provide personal information that they would only give to trusted sites. Repercussions of such attacks include “identity theft, credit card fraud, ransomware attacks, data breaches, and huge financial losses for individuals and corporations” [10].

There are multiple types of phishing attacks, namely spear phishing, clone phishing, link manipulation, website forgery, covert redirect, and voice phishing [11]. This study focuses on combating a composition of spear phishing, link manipulation, and website forgery, where a hacker sends a deceptive/manipulated link to a user to redirect them to a forged website.

II. BACKGROUND RESEARCH

Having a system in place that can effectively detect and alert users of phishing attacks is patently beneficial to all users with an email address. This technology would be especially useful to those with a lack of digital literacy, as they are the most susceptible to such attacks.



Anti-phishing toolbars

Current solutions to this issue prompt users to be more aware of phishing attacks and to download an anti-phishing toolbar [2]. Anti-phishing toolbars work by comparing websites the user visits to known phishing sites in their database. An obvious flaw with this approach is the occurrence of a website that doesn't fall in the database of this tool. This would render this method useless, leaving the victim prone to losing personal information.

Blacklisted Domains

Similar to the anti-phishing toolbar, a blacklist approach to phishing sites is suggested in [3], where lexical features from a suspicious domain are compared to those of blacklisted ones. Based on this comparison, a conclusion is drawn as to whether the site is phishing or not. The issue with this approach is the fact that the list of blacklisted domains, though extensive, will become outdated if not updated regularly. Additionally, if hackers acquire such a database, workaround domains can be created and widespread, ultimately having an adverse effect on this method. This blacklist approach proves an f1 score of 0.84, which we aim to surpass.

III. METHODS

Dataset

We obtained our dataset from the UCI Machine Learning Repository [7] which includes 30 features with 11,055 data points each. The feature explanation document can be viewed here [8]. Some of these features include “URL_Length”, “pop-UpWindow”, and “double_slash_redirecting”. Each feature has

one of three numeric values: 0 - suspicious, 1 - legitimate, or -1 - phishing. These values were initially a different format so a unicode decoding function had to be applied on the entire dataset to get the feature values in numerical format. The label distribution of a random sample of 500 data points of the example features named above are visualized in Fig 1.

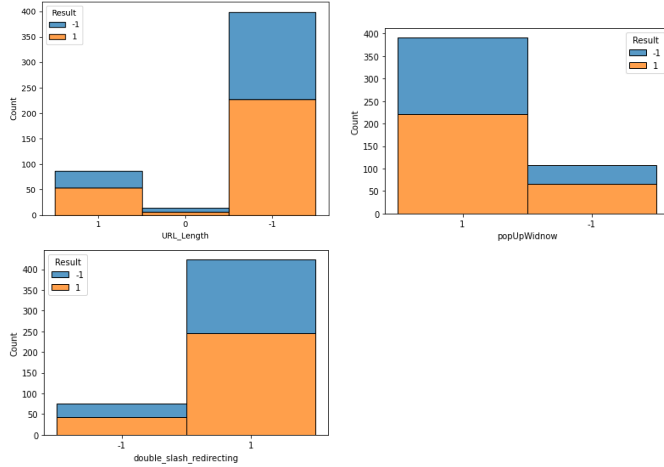


Fig. 1.

As seen in the visualizations, not all features have the 0 numerical value, but all have 1 and -1. The distribution of the labels is visualized in Fig 2.

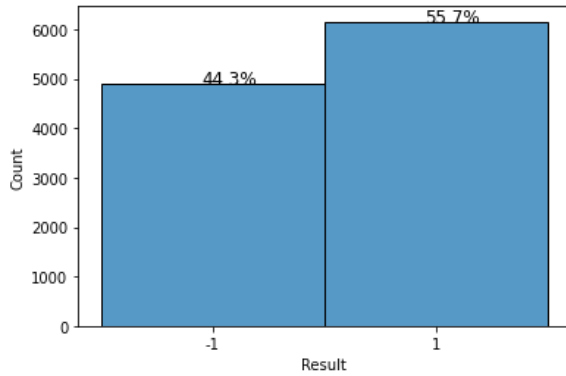


Fig. 2.

We utilized sklearn's train_test_split function to issue a 75% / 25% split between the training and testing data respectively. This resulted in the training features having: 8291 rows x 30 columns, testing features: 2764 rows x 30 columns, training labels: 8291 rows x 1 column, testing labels: 2764 rows x 1 column.

Models

Our approach started by the implementation of the KNN, or K-Nearest-Neighbors model. We started off with K values of 1 and 2, or 1 and 2 neighbors. These models were trained and tested on the UCI dataset. In order to improve accuracy for this model, we decided to fine tune the hyperparameters for

this model, namely the n_neighbors, weights, and algorithm parameters. Additionally, we created a python script to generate KNN models from 1-97 neighbors to find the optimal amount of neighbors for this model.

The next model tested was the RFC, or Random Forest Classifier. Just as we did with the KNN, we started with a fixed amount of estimators: 10. To further improve the accuracy of the RFC in this setting, we altered and utilized the same python script to generate RFC models from 1-97 estimators to find the optimal estimators for this model. The hyperparameters tuned for this model were n_estimators, criterion, and max_features.

The next method we used was an ensemble model. An ensemble model is a way to combine multiple models in a single prediction process for an overall higher accuracy. As a single estimator/model may be non-optimal for the specific type of classification and dataset used in this case, using an ensemble eliminates this issue. The ensemble we created was composed of 3 models: A decision tree, SVC, and an RFC. The makeup for this ensemble was purposeful. A decision tree is basic yet effective at classification while an SVC is great for separating complex data. The RFC, an ensemble of decision trees, is extremely useful for binary classification as well. The diversity of these models serves beneficial for the ensemble as a whole as the benefits of each are factored into the overall prediction, whereas cons that exist among the individual models will be ruled out as a collective prediction. The ensemble was formed by creating each of the 3 models separately, then employing a voting classifier and passing in all the models as parameters. The default voting parameter of "hard" was set to use the predicted class labels for majority rule voting.

IV. RESULTS

It is important to clearly define what results we are looking for in an ideal model. We want to find the model with the highest prediction accuracy, but precision, recall, and f1 scores will also be taken into account to follow correlations/explain the workings of the models.

KNN

For the initial KNN (neighbors 1-2) the confusion matrices are shown in Fig 3.

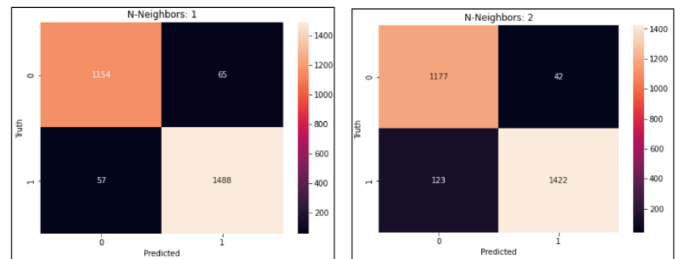


Fig. 3.

A visualization of the python script to evaluate performance metrics over neighbors can be seen in Fig 4.

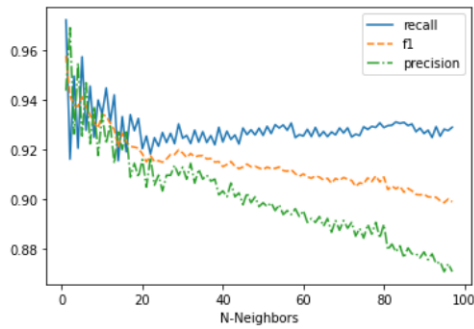


Fig. 4.

Analyzing this chart, there seems to be a noticeable decrease in all metrics after 10 neighbors, meaning a larger number of neighbors doesn't affect accuracy and other metrics positively for the KNN. Since 10 neighbors was the optimal amount for the recall, precision, and f1 scores, we kept this parameter fixed when tuning the others. The tuning of the remaining parameters is shown in Fig 5.

Weights	Algorithm	Accuracy
uniform	ball_tree	0.9340
uniform	kd_tree	0.9345
uniform	brute	0.9343
distance	ball_tree	0.9579
distance	kd_tree	0.9577
distance	brute	0.9581

Fig. 5.

RFC

The confusion matrix for the initial RFC is shown in Fig 6.

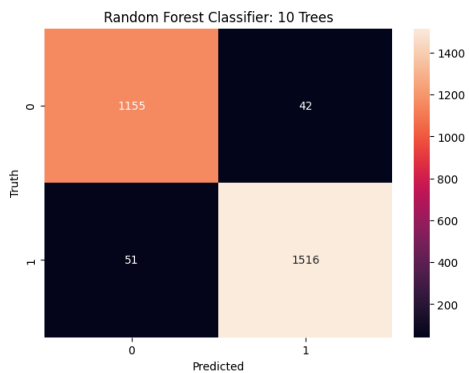


Fig. 6.

A visualization of the python script to evaluate performance metrics over estimators can be seen in Fig 7.

Contrary to the KNN, the more estimators available in the RFC, the higher its performance metrics seem to become. In fact, 100 estimators was collectively the optimal number of

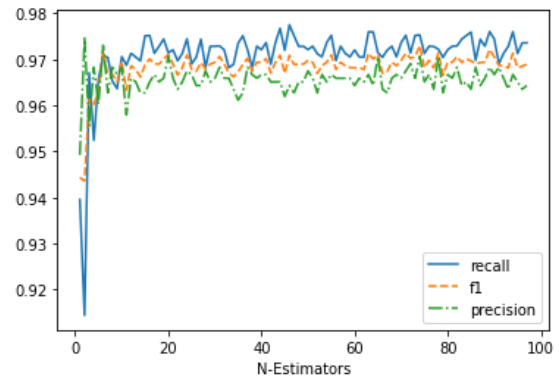


Fig. 7.

estimators for the recall, precision, and f1 scores. Hyperparameter tuning was therefore done on a base RFC with 100 estimators. The tuning of these parameters is shown in Fig 8.

Max_features	Criterion	Accuracy
sqrt	gini	0.9707
sqrt	entropy	0.9711
log2	gini	0.9706
log2	entropy	0.9708

Fig. 8.

There wasn't much variation in accuracy scores from the hyper parameter tuning, meaning that the amount of estimators was the main differentiating factor regarding accuracy. This is understandable as the RFC relies on a conglomerate of decision trees/estimators to create predictions, so the more points of predictions, the more the overall accuracy will increase.

Ensemble

The confusion matrix for the base ensemble model is shown in Fig 9.

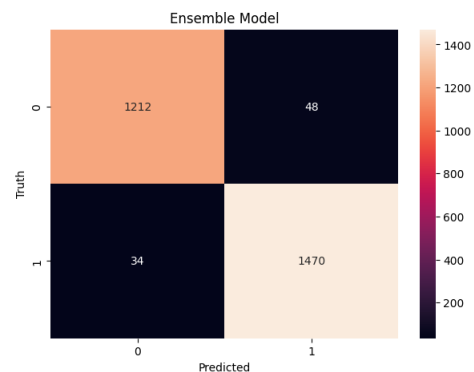


Fig. 9.

The scores for the individual models are shown in Fig 10 along with a visualization of the accuracy distributions in Fig 11.

Model	Accuracy
Decision Tree	0.9672
Random Forest Classifier	0.9681
SVC	0.9675
Ensemble	0.9671

Fig. 10.

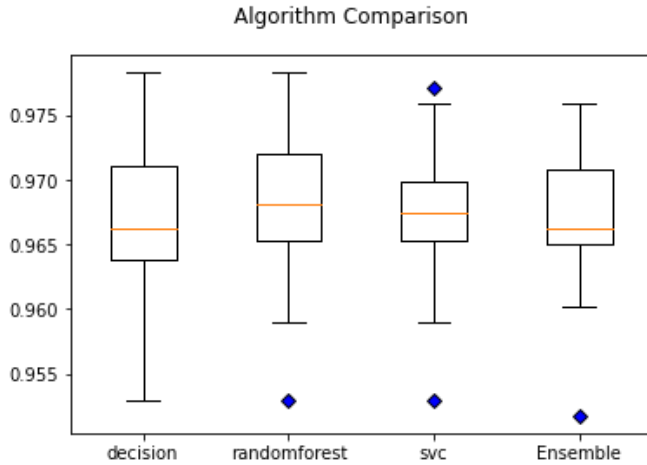


Fig. 11.

Although the RFC performed highest, the ensemble performed relatively close with a difference of only 0.001 between the accuracy scores. Additionally, no hyperparameter tuning – such as the number of models, the type of aggregation, or the weights assigned to the models – was performed to the ensemble or the Decision Tree and SVC comprising it. Overall, the ensemble ended up with an accuracy lower than all the models comprising it.

V. CONCLUSION

The Random Forest Classifier with 100 estimators, a max_features value of “sqrt”, and a criterion value of “entropy” performed the best out of all models tested, with an accuracy score of 97.11%. This accuracy was achieved after hyperparameter tuning was performed on the base RFC. Though the ensemble didn’t perform better than the tuned RFC with a score of 96.71%, it still achieved a higher accuracy score than the tuned KNN which had 95.81%. The ensemble may have performed worse than the models comprising it because when faced with conflicting predictions, it may inaccurately reconcile these conflicts, leading to lower accuracy. We believe an ensemble model could definitely outdo the RFC if tuned properly. To reiterate, the ensemble model discussed in this study was not tuned. Based on the accuracy improvements from the KNN and RFC post-tuning, we can infer that an

increase of around 1-2% would be evident if the ensemble were to be tuned optimally, boosting its accuracy score over the RFC in this case. Next steps for this research include fine tuning the discussed ensemble model, generating ensembles with a different model makeup and evaluating their results, and creating a plugin/application to apply this model to directly scan emails/sites for phishing attacks.

REFERENCES

- [1] “Creating multiple subplots using plt.subplots — Matplotlib 3.7.2 documentation.” Matplotlib, https://matplotlib.org/stable/gallery/subplots_axes_and_figures/subplots_demo.html.
- [2] “8 Simple Measures For Phishing Attack Prevention - PhishProtection.com.” Phish Protection, <https://www.phishprotection.com/content/phishing-prevention/phishing-attack-prevention>.
- [3] Hong, J., Kim, T., Liu, J., Park, N., Kim, SW. (2020). Phishing URL Detection with Lexical Features and Blacklisted Domains. In: Jajodia, S., Cybenko, G., Subrahmanian, V., Swarup, V., Wang, C., Wellman, M. (eds) Adaptive Autonomous Secure Cyber Systems. Springer, Cham. https://doi.org/10.1007/978-3-030-33432-1_12
- [4] Parekh, Shivani. “Ensemble model : Data Visualization — by Shivani Parekh — Analytics Vidhya.” Medium, 9 October 2020, <https://medium.com/analytics-vidhya/ensemble-model-data-visualization-2f4cb06859c1>.
- [5] Parekh, Shivani. “Ensemble Modelling: How it is performed in Python- A quick guide — Analytics Vidhya.” Medium, 26 August 2020, <https://medium.com/analytics-vidhya/ensemble-modelling-in-a-simple-way-386b6cbaf913>.
- [6] “Phishing Defense by Design.” Coforge, <https://www.coforge.com/resource-library/white-papers/phishing-defense-design>.
- [7] “Phishing Websites.” UCI Machine Learning Repository, 25 March 2015, <https://archive.ics.uci.edu/dataset/327/phishing+websites>.
- [8] Phishing Websites Features, <https://eprints.hud.ac.uk/id/eprint/24330/6/MohammadPhish>
- [9] “Plot Multiple lines in Matplotlib.” GeeksforGeeks, 25 November 2020, <https://www.geeksforgeeks.org/plot-multiple-lines-in-matplotlib/>.
- [10] “What is a phishing attack?” IBM, <https://www.ibm.com/topics/phishing>.
- [11] “What is a Phishing attack? — mlytics.” Mlytics Learning Center, <https://learning.mlytics.com/cyber-attacks/what-is-a-phishing-attack/>.